

南京大学本科生实验报告

课程名称：计算机网络

任课教师：李文中

助教：

学院	计算机科学与技术	专业（方向）	计算机科学与技术
学号	221220041	姓名	徐庶
Email	3034819436@qq.com	开始/完成日期	2024.03.14-2024.03.17

一、实验名称

Lab 1: Switchyard & Mininet

二、实验目的

对计算机网络实验所需的工具进行初步的学习和使用

三、实验内容

首先配置实验所需环境，包括 Linux，python，git 等，然后通过五个 question 来熟悉 mininet 拓扑网络构建，Wireshark 抓包等操作

四、实验结果

Step 1: Modify the Mininet topology

进入 start_mininet 文件，在创建 node 的字典中直接把 server2 节点删除即可。不过在 step4 中需要构建一个新的拓扑网络，于是乎我就增加了一个新的节点：client2，然后仿照已有节点设置其 MAC 和 IP 地址。

于是形成了如下拓扑：

```
# Host and link configuration
#
#
#   server1
#       \
#        hub----client
#       /
#   client2
#
```

通过 mininet 的相关指令可以验证其构建成功：

```
*** Starting CLI:
mininet> nodes
available nodes are:
client client2 hub server1
mininet> net
client client-eth0:hub-eth1
client2 client2-eth0:hub-eth0
hub hub-eth0:client2-eth0 hub-eth1:client-eth0 hub-eth2:server1-eth0
server1 server1-eth0:hub-eth2
mininet> dump
<Host client: client-eth0:192.168.100.3 pid=5562>
<Host client2: client2-eth0:192.168.100.4 pid=5564>
<Host hub: hub-eth0:10.0.0.3,hub-eth1:None,hub-eth2:None pid=5566>
<Host server1: server1-eth0:192.168.100.1 pid=5568>
mininet>
```

Step 2: Modify the logic of a device

在 myhub.py 中，可以发现有两个函数：net.recv_packet()和 net.send_packet(intf, packet)，大概可以猜测到，他们分别表示接收和发送了一个 packet。那么只需要定义两个变量：

num_in,num_out,分别统计两个函数执行的次数，就可以获得出入包的数量，最后打印即可。
使用 pingall 制造流量并在 hub 中打印其信息后结果如下：

```
njucs@njucs-VirtualBox: ~/lab-1-G221220041
File Edit View Search Terminal Help

('server1', <TCIntf server1
('hub', <TCIntf hub-eth0>,
('hub', <TCIntf hub-eth1>,
0:00:00:01 IP | IPv4 192.168.100.3->192.168.100.1 ICMP | ICMP EchoRequest 2838
(56 data bytes) to hub-eth1
*** client : ('sysctl -w net
13:54:42 2024/03/17 INFO in:3 out:3
net.ipv6.conf.all.disable_
13:54:43 2024/03/17 INFO Flooding packet Ethernet 10:00:00:00:01->30:00:
0:00:00:01 IP | IPv4 192.168.100.1->192.168.100.3 ICMP | ICMP EchoReply 2838 1
*** client : ('sysctl -w net
56 data bytes) to hub-eth0
net.ipv6.conf.default.disable_
13:54:43 2024/03/17 INFO in:4 out:4
*** hub : ('sysctl -w net.i
13:54:43 2024/03/17 INFO Flooding packet Ethernet 10:00:00:00:01->30:00:
0:00:00:01 IP | IPv4 192.168.100.1->192.168.100.3 ICMP | ICMP EchoRequest 2841
net.ipv6.conf.all.disable_i
(56 data bytes) to hub-eth0
*** hub : ('sysctl -w net.i
13:54:43 2024/03/17 INFO in:5 out:5
net.ipv6.conf.default.disable_
13:54:43 2024/03/17 INFO Flooding packet Ethernet 30:00:00:00:01->10:00:
0:00:00:01 IP | IPv4 192.168.100.3->192.168.100.1 ICMP | ICMP EchoReply 2841 1
*** server1 : ('sysctl -w n
56 data bytes) to hub-eth1
net.ipv6.conf.all.disable_i
13:54:43 2024/03/17 INFO in:6 out:6
*** server1 : ('sysctl -w n
13:54:48 2024/03/17 INFO Flooding packet Ethernet 10:00:00:00:01->30:00:
0:00:00:01 ARP | Arp 10:00:00:00:00:01:192.168.100.1 00:00:00:00:00:00:192.168.
00.3 to hub-eth0
net.ipv6.conf.default.disable_
13:54:48 2024/03/17 INFO in:7 out:7
*** Starting controller
13:54:48 2024/03/17 INFO Flooding packet Ethernet 30:00:00:00:01->10:00:
0:00:00:01 ARP | Arp 30:00:00:00:00:01:192.168.100.3 10:00:00:00:00:01:192.168.
00.1 to hub-eth1
13:54:48 2024/03/17 INFO in:8 out:8

*** Starting 0 switches

*** Starting CLI:
mininet> xterm hub
mininet> pingall
*** Ping: testing ping reachability
client -> X server1
hub -> X X
server1 -> client X
*** Results: 66% dropped (2/6 received)
mininet> 
```

Step 3: Modify the test scenario of a device

在 testcases/myhub_testscenario.py 文件中有一些测试样例，阅读之后可以仿写一个，我写了一个当源 MAC 为全 F，即源是广播时的情况。此时该帧其实是无意义的，甚至是带有攻击性的，所以 hub 此时应该啥也不做。

使用 swyard -t ./testcases/myhub_testscenario.py myhub.py 指令进行测试，能够通过所有的测试用例。

```

(syenv) njucs@njucs-VirtualBox:~/lab-1-G221220041$ swyard -t ./testcases/myhub_testscenario.py myhub.py
15:41:10 2024/03/17 INFO Starting test scenario ./testcases/myhub_testscenario.py
15:41:10 2024/03/17 INFO Flooding packet Ethernet 30:00:00:00:00:02->ff:ff:ff:ff:ff:ff
:ff IP | IPv4 172.16.42.2->255.255.255.255 ICMP | ICMP EchoRequest 0 0 (0 data bytes) to
o eth0
15:41:10 2024/03/17 INFO in:1 out:1
15:41:10 2024/03/17 INFO Flooding packet Ethernet 30:00:00:00:00:02->ff:ff:ff:ff:ff:ff
:ff IP | IPv4 172.16.42.2->255.255.255.255 ICMP | ICMP EchoRequest 0 0 (0 data bytes) t
o eth2
15:41:10 2024/03/17 INFO in:1 out:2
15:41:10 2024/03/17 INFO Flooding packet Ethernet 20:00:00:00:00:01->30:00:00:00:00
:02 IP | IPv4 192.168.1.100->172.16.42.2 ICMP | ICMP EchoRequest 0 0 (0 data bytes) to
eth1
15:41:10 2024/03/17 INFO in:2 out:3
15:41:10 2024/03/17 INFO Flooding packet Ethernet 20:00:00:00:00:01->30:00:00:00:00
:02 IP | IPv4 192.168.1.100->172.16.42.2 ICMP | ICMP EchoRequest 0 0 (0 data bytes) to
eth2
15:41:10 2024/03/17 INFO in:2 out:4
15:41:10 2024/03/17 INFO Flooding packet Ethernet 30:00:00:00:00:02->20:00:00:00:00
:01 IP | IPv4 172.16.42.2->192.168.1.100 ICMP | ICMP EchoReply 0 0 (0 data bytes) to et
h0
15:41:10 2024/03/17 INFO in:3 out:5
15:41:10 2024/03/17 INFO Flooding packet Ethernet 30:00:00:00:00:02->20:00:00:00:00
:01 IP | IPv4 172.16.42.2->192.168.1.100 ICMP | ICMP EchoReply 0 0 (0 data bytes) to et
h2
15:41:10 2024/03/17 INFO in:3 out:6
15:41:10 2024/03/17 INFO Received a packet intended for me
15:41:11 2024/03/17 INFO Received a packet intended for me

Results for test scenario hub tests: 10 passed, 0 failed, 0 pending
Passed:
1 An Ethernet frame with a broadcast destination address
  should arrive on eth1
2 The Ethernet frame with a broadcast destination address
  should be forwarded out ports eth0 and eth2
3 An Ethernet frame from 20:00:00:00:00:01 to
  30:00:00:00:00:02 should arrive on eth0
4 Ethernet frame destined for 30:00:00:00:00:02 should be
  flooded out eth1 and eth2
5 An Ethernet frame from 30:00:00:00:00:02 to
  20:00:00:00:00:01 should arrive on eth1
6 Ethernet frame destined to 20:00:00:00:00:01 should be
  flooded out eth0 and eth2
7 An Ethernet frame should arrive on eth2 with destination
  address the same as eth2's MAC address
8 The hub should not do anything in response to a frame
  arriving with a destination address referring to the hub
  itself.
9 An Ethernet frame should arrive on eth2 with destination
  address the same as eth2's MAC address
10 The hub should not do anything in response to a frame whose
  hwsrc is broadcast

All tests passed!

```

不过在实验过程中，我将样例三中的 PacketInputEvent 函数的第一个参数由 eth2 改为 eth1，运行后样例依然通过，不太明白发生甚莫事情了。

Step 4: Run your device in Mininet

在 step1 中有详细说明，具体验证流程为：先启动 start_mininet，该文件配置了四个节点：server1, hub, client 和 client2。随后通过 xterm 进入虚拟环境启动 hub，在运行 myhub 用来监控流量，这时候在 mininet 中 pingall 制造流量就能看到各个节点间的数据转发了。

```
njucs@njucs-VirtualBox: ~/lab-1-G221220041
File Edit View Search Terminal Help
*** server1 : ('sysctl -w net.ipv6.conf.all.disable_ipv6 = 1
net.ipv6.conf.default.disable_ipv6 = 1
*** Starting controller
*** Starting 0 switches
*** Starting CLI:
mininet> xterm hub
mininet> pingall
*** Ping: testing ping reachability
client -> client2 X server1
client2 -> client X server1
hub -> X X X
server1 -> client client2 X
*** Results: 50% dropped (6/12 receive
mininet> node
*** Unknown command: node
mininet> nodes
available nodes are:
client client2 hub server1
mininet> net
client client-eth0:hub-eth1
client2 client2-eth0:hub-eth0
hub hub-eth0:client2-eth0 hub-eth1:client-eth0 hub-eth2:server1-eth0
server1 server1-eth0:hub-eth2
mininet> █
```

"Node: hub"

16:01:01	2024/03/17	INFO	Flooding packet Ethernet 10:00:00:0
	Arp 10:00:00:00:00:01:192.168.100.1	00:00:00:00:00:00:192.168.	
16:01:01	2024/03/17	INFO	in:21 out:41
16:01:01	2024/03/17	INFO	Flooding packet Ethernet 10:00:00:
	Arp 10:00:00:00:00:01:192.168.100.1	00:00:00:00:00:00:192.168	
16:01:01	2024/03/17	INFO	in:21 out:42
16:01:02	2024/03/17	INFO	Flooding packet Ethernet 30:00:00:
	Arp 30:00:00:00:00:01:192.168.100.3	10:00:00:00:00:01:192.168	
16:01:02	2024/03/17	INFO	in:22 out:43
16:01:02	2024/03/17	INFO	Flooding packet Ethernet 30:00:00:
	Arp 30:00:00:00:00:01:192.168.100.3	10:00:00:00:00:01:192.168	
16:01:02	2024/03/17	INFO	in:22 out:44
16:01:03	2024/03/17	INFO	Flooding packet Ethernet 10:00:00:
	Arp 10:00:00:00:00:01:192.168.100.1	00:00:00:00:00:00:192.168	
16:01:03	2024/03/17	INFO	in:23 out:45
16:01:03	2024/03/17	INFO	Flooding packet Ethernet 10:00:00:
	Arp 10:00:00:00:00:01:192.168.100.1	00:00:00:00:00:00:192.168	
16:01:03	2024/03/17	INFO	in:23 out:46
16:01:03	2024/03/17	INFO	Flooding packet Ethernet 50:00:00:
	Arp 50:00:00:00:00:01:192.168.100.4	10:00:00:00:00:01:192.168	
16:01:03	2024/03/17	INFO	in:24 out:47
16:01:03	2024/03/17	INFO	Flooding packet Ethernet 50:00:00:
	Arp 50:00:00:00:00:01:192.168.100.4	10:00:00:00:00:01:192.168	
16:01:03	2024/03/17	INFO	in:24 out:48

Step 5: Capture using Wireshark

首先通过指令 `client wireshark &` 用 wireshark 对 client 的 eth0 端口进行抓包

```
client-eth0
```

之后让 client ping server1 制造流量进行抓包，抓取一段时间后用 `ctrl-c` 终止 ping 的进程。

```
*** Starting CLI:
mininet> xterm hub
mininet> client wireshark &
mininet> client ping server1
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-root'
PING 192.168.100.1 (192.168.100.1) 56(84) bytes of data.
64 bytes from 192.168.100.1: icmp_seq=1 ttl=64 time=965 ms
64 bytes from 192.168.100.1: icmp_seq=2 ttl=64 time=489 ms
64 bytes from 192.168.100.1: icmp_seq=3 ttl=64 time=437 ms
64 bytes from 192.168.100.1: icmp_seq=4 ttl=64 time=428 ms
^C
--- 192.168.100.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3027ms
rtt min/avg/max/mdev = 428.255/580.098/965.463/223.722 ms
mininet> █
```

在 wireshark 中可以看到我们抓取了如下数据包：

Time	Source	Destination	Protocol	Length	Info
1 0.000000000	30:00:00:00:00:01	Broadcast	ARP	42	Who has 192.168.100.1?
2 0.445948496	Private_00:00:01	30:00:00:00:00:01	ARP	42	192.168.100.1 is at 10
3 0.546608711	192.168.100.3	192.168.100.1	ICMP	98	Echo (ping) request i
4 0.857544765	192.168.100.1	192.168.100.3	ICMP	98	Echo (ping) reply i
5 0.995017279	192.168.100.3	192.168.100.1	ICMP	98	Echo (ping) request i
6 1.381936660	192.168.100.1	192.168.100.3	ICMP	98	Echo (ping) reply i
7 1.994028226	192.168.100.3	192.168.100.1	ICMP	98	Echo (ping) request i
8 2.330384115	192.168.100.1	192.168.100.3	ICMP	98	Echo (ping) reply i
9 3.022231891	192.168.100.3	192.168.100.1	ICMP	98	Echo (ping) request i

▶ Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0
 ▶ Ethernet II, Src: 30:00:00:00:00:01 (30:00:00:00:00:01), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
 ▶ Address Resolution Protocol (request)

Client 首先进行了广播并询问谁的 ip 地址是 192.168.100.1，并得到了 server1 返回的包，两者都遵从 ARP 协议，建立连接后，就开始不断的 request 和 reply，在数据传输过程中还能看到很多详细信息：

▶ Flags: 0x0000 ...0 0000 0000 0000 = Fragment offset: 0 Time to live: 64 Protocol: ICMP (1) Header checksum: 0x506d [validation disabled] [Header checksum status: Unverified] Source: 192.168.100.1 Destination: 192.168.100.3
▼ Internet Control Message Protocol Type: 0 (Echo (ping) reply) Code: 0 Checksum: 0x4fe0 [correct] [Checksum Status: Good] Identifier (BE): 4110 (0x100e) Identifier (LE): 3600 (0x0e10) Sequence number (BE): 3 (0x0003) Sequence number (LE): 768 (0x0300) [Request frame: 7] [Response time: 336.356 ms] Timestamp from icmp data: Mar 17, 2024 16:12:53.000000000 CST [Timestamp from icmp data (relative): 1.039190229 seconds]
▼ Data (48 bytes) Data: dc2f090000000000101112131415161718191a1b1c1d1e1f... [Length: 48]

如他们遵从的协议，包的源 ip 和目的 ip，响应时间，48 字节的数据内容等。

五、核心代码

1.start_mininet 文件中增删的节点：

```
nodes = {
    "server1": {
        "mac": "10:00:00:00:00:{:02x}",
        "ip": "192.168.100.1/24"
    },
    "client": {
        "mac": "30:00:00:00:00:{:02x}",
        "ip": "192.168.100.3/24"
    },
    "client2": {
        "mac": "50:00:00:00:00:{:02x}",
        "ip": "192.168.100.4/24"
    },
    "hub": {
        "mac": "40:00:00:00:00:{:02x}",
    }
}
```

2.myhub.py 中统计包的出入量：

```

num_in = 0
num_out = 0

while True:
    try:
        _, fromIface, packet = net.recv_packet()
        num_in+=1
    except NoPackets:
        continue
    except Shutdown:
        break

    log_debug (f"In {net.name} received packet {packet} on {fromIface}")
    eth = packet.get_header(Ethernet)
    if eth is None:
        log_info("Received a non-Ethernet packet?!")
        return
    if eth.dst in mymacs:
        log_info("Received a packet intended for me")
    else:
        for intf in my_interfaces:
            if fromIface!= intf.name:
                log_info (f"Flooding packet {packet} to {intf.name}")
                net.send_packet(intf, packet)
                num_out+=1
                log_info("in:{} out:{}".format(num_in,num_out))

```

3. myhub_testscenario.py 中自定义的测试用例

```

# test case 4: a meaningless frame should result in nothing happening
reqpkt = new_packet(
    "ff:ff:ff:ff:ff:ff",
    "10:00:00:00:00:03",
    '192.168.1.100',|
    '172.16.42.2'
)
s.expect(
    PacketInputEvent("eth2", reqpkt,display=Ethernet),
    ("An Ethernet frame should arrive on eth2 with destination address "
    "the same as eth2's MAC address")
)
s.expect(
    PacketInputTimeoutEvent(1.0),
    ("The hub should not do anything in response to a frame whose hwsrc"
    " is broadcast")
)

```

六、总结与感想

总结：学会了使用 git（好耶，并熟悉了用于计网的各个工具的使用方法和具体功能

感想：第一次实验蒙蔽的我是一脸的蒙蔽，不知道自己该干什么，只能对着实验手册一遍一遍地阅读，而且实验手册是英文的，看着也很吃力，好在第一次实验比较简单，不过最后实验结束了也不太能确定自己是不是做完了做对了……